

A Drift-Stable Quantum Federated Learning for Intelligent Services

Shanika Iroshi Nanayakkara and Shiva Raj Pokhrel, *Senior IEEE Member*

Abstract—Quantum federated learning enables distributed clients to train quantum neural networks without sharing local data, making it promising for privacy-aware intelligent services. Intelligent services in this context refer to privacy-sensitive distributed decision systems, such as fraud detection and genomic classification, where reliable and fair client-level learning is as important as the accuracy of the aggregate model. However, heterogeneous client data and noisy quantum optimization often cause unstable local updates, client drift, and unfair performance between clients. This paper proposes DUQFL-Prox, a drift-stable quantum federated learning framework based on deep-unfolded local optimization. Instead of using a fixed local optimizer, each client performs adaptive unfolded SPSA updates, while a proximal term keeps the local model close to the global model. A lightweight controller learns step-specific optimization parameters to improve post-aggregation performance. Experiments on financial fraud and genomic classification tasks show that DUQFL-Prox improves stability, generalization, and client fairness compared with standard QFL baselines. The results suggest that deep-unfolded quantum federated learning can support more reliable and fair intelligent services in heterogeneous distributed environments.

Index Terms—Quantum machine learning, Client drift, Heterogeneity quantum federated learning, Deep unfolding, Learning-to-learn, Hyperparameter tuning

I. INTRODUCTION

FEDERATED learning (FL) has emerged as a promising framework for collaborative model training, where multiple clients optimize a shared model without centralizing raw local data. By allowing clients to train locally and communicate only model updates to a coordinating server, FL provides an attractive paradigm for privacy-aware and communication-efficient intelligence [1]. As quantum machine learning continues to develop, this distributed setting becomes increasingly relevant in quantum contexts as well, giving rise to quantum federated learning (QFL), in which multiple clients collaboratively train quantum or quantum-enhanced models while keeping local data decentralized [2]–[4]. In this setting, however, optimization becomes substantially more challenging than in conventional classical FL, because local training of quantum neural networks (QNNs) is often noisy, nonconvex, shot-sensitive, and highly dependent on optimizer configuration [2], [5].

In this work, the phrase “intelligent services” refers to distributed AI-enabled service environments in which data-driven decisions must be learned from decentralized, privacy-sensitive, and heterogeneous client data. Such services include financial fraud detection, genomic classification, healthcare analytics, cyber-physical monitoring [6], and edge intelligence,

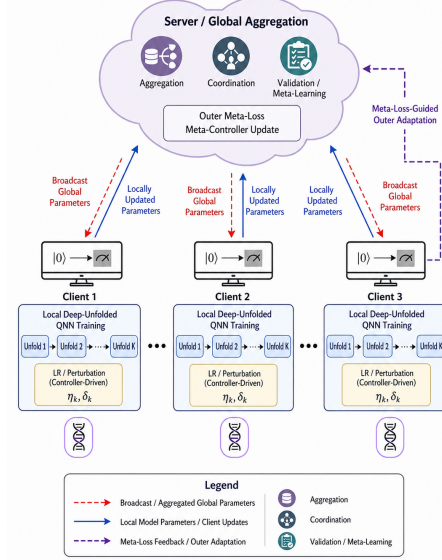


Fig. 1: High-level DUQFL setup with adaptive local QNN training, selected-model aggregation, and controller-guided global optimization.

where data are naturally distributed across institutions, devices, or service providers and cannot be freely centralized due to privacy, regulatory, or ownership constraints [7]. These settings require not only high aggregate accuracy, but also stable optimization, fair client-level performance, and reliable generalization across heterogeneous participants [8].

In heterogeneous federated environments, one common strategy for improving training efficiency is to exclude, down-weight, delay, or selectively sample slow clients, often referred to as stragglers. Previous FL studies have shown that system heterogeneity, including differences in computation and communication capabilities, can substantially slow synchronous federated training [1], [9], [10]. Client-selection methods such as Oort therefore improve time-to-accuracy by prioritizing clients with favorable statistical and system utility [11]. However, selection or straggler-mitigation strategies can also create a representativeness issue when slow, resource-limited, or statistically distinctive clients are repeatedly underrepresented. Recent work on biased client selection and participation imbalance shows that non-uniform client participation can bias the learned global model and affect fairness or client-level performance [12], [13]. In practical QFL settings, this issue is undesirable because heterogeneous clients may contain rare, domain-specific, or clinically important data distributions. Rather than excluding such clients, DUQFL-Prox aims to stabilize their local QNN optimization trajectories so that their updates remain useful for global aggregation.

Shanika Iroshi Nanayakkara is with School of IT, Deakin University, VIC 3125, Burwood, Australia, (e-mail: s222112938@deakin.edu.au).

Shiva Raj Pokhrel is with School of IT, Deakin University, VIC 3125, Burwood, Australia, (e-mail: shiva.pokhrel@deakin.edu.au).

A central limitation in existing QFL pipelines is that local optimization is often treated as a fixed procedure. In a typical setting, each client receives the broadcast global model, performs a predetermined number of local optimization steps using a fixed optimizer schedule, and returns the resulting parameters to the server. This design implicitly assumes that the same local training rule is suitable for all clients, data distributions, and communication rounds. Such an assumption is restrictive in QFL, where QNN optimization is sensitive to learning rate, perturbation scale, measurement noise, and the non-convex geometry of variational quantum loss landscapes. In non-IID data, fixed local optimization can therefore produce uneven client updates, excessive client drift, and unstable post-aggregation behavior.

SPSA is attractive for QNN training because it estimates an update direction using only two objective-function evaluations, independent of the number of trainable parameters [14]. This property makes SPSA practical for variational quantum models, particularly when analytic gradients are costly, unavailable, or noisy [15]. However, in federated QNN training, a fixed SPSA schedule can be too rigid. Conservative settings may slow local improvement, whereas aggressive settings may amplify client drift and reduce the compatibility of local updates during aggregation. Therefore, the central challenge is not merely to use SPSA, but to make local SPSA-based QNN optimization adaptive while keeping client updates aligned with the global federated objective.

To address this challenge, we propose *Deep-Unfolded Quantum Federated Learning with Proximal Regularization (DUQFL-Prox)*, a drift-stable QFL framework for heterogeneous intelligent services. DUQFL-Prox decomposes local SPSA-based QNN training into K unfolded optimization blocks. At each unfold step, a lightweight shared controller generates step-specific SPSA learning rates and perturbation scales from optimization-state features, including unfold progress, recent loss behavior, parameter displacement, and client context. This enables local QNN optimization to adapt across clients, unfold steps, and communication rounds, rather than relying on a fixed optimizer schedule.

However, adaptive local optimization alone can amplify client drift under non-IID federated data. DUQFL-Prox therefore introduces proximal regularization into the client objective to penalize excessive deviation from the broadcast global model. This encourages each client to improve their local QNN objective while maintaining aggregation-compatible updates. In addition, each client uploads the validation-preferred unfolded checkpoint rather than necessarily returning the final unfolded state, preventing over-aggressive later updates from degrading global aggregation.

DUQFL-Prox follows a bilevel learning structure. At the inner level, each client performs deep-unfolded proximal SPSA optimization of its local QNN parameters. At the outer level, the shared controller is periodically refined using a meta-objective defined on post-aggregation validation behaviour. Thus, DUQFL-Prox learns not only the QNN parameters but also how local QNN optimization should evolve so that client updates become more stable, fair, and useful after aggregation.

The main contributions of this work are summarized as

follows:

- We propose DUQFL-Prox, a drift-stable deep-unfolded QFL framework for heterogeneous intelligent services, where local QNN training is modeled as an adaptive multi-step optimization process rather than a fixed optimizer routine.
- We design a controller-driven SPSA mechanism that generates unfold-specific learning rates and perturbation scales from local optimization-state features, enabling adaptive QNN training across clients, communication rounds, and unfolded steps.
- We incorporate proximal regularization and validation-based best-unfold selection to reduce client drift, prevent over-specialized local updates, and improve aggregation compatibility.

Importantly, we introduce an outer meta-loss-guided controller adaptation mechanism that aligns local optimization behavior with post-aggregation global performance. We evaluate DUQFL-Prox across financial fraud detection and genomic classification tasks using global accuracy, client-level generalization, train–test gap, fairness gap, and imbalance-aware classification metrics.

The remainder of this paper is organized as follows. Section II reviews related work on QFL, QNN optimization, federated heterogeneity, and deep unfolding. Section III presents the problem formulation and the DUQFL-Prox methodology. Section IV describes the experimental setup and reproducibility protocol. Section V presents the empirical results and ablation analysis. Section ?? discusses implications, limitations, and future research directions, and Section VI concludes the paper.

II. RELATED WORK

Table I demonstrates that prior QFL studies have mainly focused on secure quantum communication, privacy-preserving learning, application-specific QFL implementations, and server-side aggregation mechanisms [3], [16]–[19], [25]–[29]. While these studies establish the feasibility of QFL. However, local QNN optimization is typically treated as a fixed inner routine. This assumption becomes restrictive when client data are non-independent and identically distributed (non-IID) and quantum measurements are stochastic. In such cases, local updates may become unstable, diverge from the global model, and reduce client-level generalization.

QNN training commonly relies on optimizers such as gradient descent, Adam, COBYLA, and SPSA. SPSA is particularly suitable for variational quantum models because it estimates a stochastic update direction using only two objective function evaluations, independent of the number of trainable parameters [33], [34]. However, in federated QNN training, a fixed SPSA schedule may be inadequate: conservative settings can slow local improvement, whereas aggressive settings can amplify client drift and reduce aggregation compatibility. Therefore, the key challenge is not merely to use SPSA, but to adapt SPSA-based local QNN optimization while preserving global federated consistency.

Deep unfolding provides a principled way to convert iterative optimization into a structured and learnable update

TABLE I: Research gap. $\checkmark$ indicates *explicit support*, $\triangle$ indicates *partial support*, and $\times$ indicates *limited or no support*.

Research stream	References	QFL	QNN	Adaptive	Client drift	L2L	Generalization	Observed Gap
Secure / privacy-preserving QFL	[16]–[19]	$\checkmark$	$\triangle$	$\times$	$\times$	$\times$	$\times$	Focuses mainly on privacy/security rather than optimiser stability or heterogeneous client drift.
QFL for financial fraud detection	[20]	$\checkmark$	$\checkmark$	$\triangle$	$\triangle$	$\times$	$\triangle$	Applies quantum federated neural networks to financial fraud detection, but does not explicitly formulate local QNN optimization as a deep-unfolded SPSA trajectory with proximal client-drift control or outer meta-loss-guided controller adaptation.
Quantum-secured fraud detection / blockchain-assisted quantum applications	[21]	$\times$	$\triangle$	$\times$	$\times$	$\times$	$\triangle$	Studies quantum-state verification, blockchain-assisted verification, and quantum-enhanced fraud detection, but does not address federated QNN optimization, client drift, or learning-to-learn.
Post-quantum data integrity and secure outsourcing	[22]	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	Addresses post-quantum secure data outsourcing and public integrity verification for cloud storage, but does not focus on QFL, QNN training, or heterogeneous federated optimization.
QFL for autonomous / distributed systems	[23], [24]	$\checkmark$	$\checkmark$	$\triangle$	$\triangle$	$\times$	$\triangle$	Studies distributed or application-specific QFL, but local optimiser trajectories are not meta-controlled.
QFL for healthcare and biomedical learning	[3], [25], [26]	$\checkmark$	$\checkmark$	$\triangle$	$\triangle$	$\times$	$\triangle$	Demonstrates application feasibility, but Often relies on global accuracy without detailed client-level analysis.
Quantum natural-gradient / optimization-based QFL	[27]–[29]	$\checkmark$	$\checkmark$	$\triangle$	$\triangle$	$\times$	$\triangle$	Improves local quantum optimization but is usually not designed for federated non- IID settings.
Classical privacy-preserving FL with adaptive pruning	[30]	$\times$	$\times$	$\checkmark$	$\triangle$	$\times$	$\triangle$	Addresses privacy-preserving FL, adaptive model pruning, communication cost, and non-IID data in classical federated learning, but does not consider QNN training, quantum optimization, deep unfolding, or quantum client drift.
Classical deep unfolding / learning-to-optimize	[31], [32]	$\times$	$\times$	$\checkmark$	$\triangle$	$\checkmark$	$\triangle$	Provides learnable optimisation principles, but is not designed for quantum neural networks or federated QNN training.
Proposed DUQFL-Prox	This work	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	Deep-unfolded SPSA-based local QNN optimization with proximal client-drift regularisation, outer/meta validation guidance, and multi-domain evaluation using global accuracy, client accuracy, fairness, and generalization metrics.

process [35]. In parallel, classical FL methods such as FedProx and SCAFFOLD show that client drift is a central obstacle under heterogeneous data and that proximal or correction-based mechanisms can improve stability [1], [36]. Although these methods are not quantum-specific, they motivate the development of adaptive and drift-aware local optimization strategies for QFL.

Motivated by these gaps, we propose *Deep-Unfolded Quantum Federated Learning with Proximal Regularization (DUQFL-Prox)*. DUQFL-Prox unfolds local SPSA-based QNN training into multiple controller-guided optimization blocks. The controller generates step-specific learning rates and perturbation scales from local optimization-state features, while a proximal term penalizes excessive deviation from the broadcast global model. The controller is further refined using a post-aggregation meta-objective, linking local optimization behaviour to global validation performance.

The proposed framework differs from prior QFL work in three aspects. First, local QNN optimization is modeled as a controller-driven unfolded process rather than a fixed client routine. Second, proximal regularization is incorporated to improve aggregation compatibility under client drift. Third, the controller is updated using post-aggregation validation behaviour, rather than relying only on local training signals. To the best of our knowledge, this combination of unfolded SPSA-based QNN optimization, proximal client-drift control, and outer meta-loss-guided controller adaptation has not been explicitly developed in existing QFL literature.

Unlike client-filtering or straggler-removal strategies, DUQFL-Prox does not discard heterogeneous clients. Instead, it stabilizes their local optimization trajectories so that their updates remain useful for aggregation. This is important in biomedical, genomic, fraud-detection, and remote-sensing

settings, where difficult clients may contain rare but important patterns. Accordingly, our evaluation reports global accuracy together with mean client test accuracy, train–test gap, client fairness gap, and imbalance-aware metrics such as F1-score, precision, recall, ROC-AUC, PR-AUC, and MCC.

III. PROBLEM FORMULATION AND METHODOLOGY

A. Problem Statement

The objective of this work is to improve the stability, generalization, and client-level reliability of QFL under heterogeneous client participation. In conventional QFL, each client receives the broadcast global QNN parameters, performs local optimization, and returns the updated parameters for server aggregation. However, under non-IID data and stochastic quantum measurements, fixed local optimization can produce unstable client trajectories and excessive drift from the global model. Consequently, the aggregated model may achieve reasonable global accuracy while still exhibiting poor client-level generalization or imbalanced performance across clients.

Although adaptive optimizers such as Adam can adjust local parameter updates, they are not designed to explicitly account for federated client heterogeneity, quantum measurement noise, post-aggregation behaviour, or drift from the broadcast global model [37]. Similarly, straggler removal or client down-weighting can improve training efficiency, but may reduce representativeness by underutilizing clients with limited resources or statistically distinctive data [1], [9], [10]. This is undesirable in QFL applications such as genomics, biomedical analysis, fraud detection, and remote sensing, where difficult clients may contain rare but important data patterns.

DUQFL-Prox addresses this problem by regulating local QNN optimization rather than excluding heterogeneous

clients. The proposed method seeks a QFL training procedure in which:

- 1) local SPSA-based QNN updates adapt across unfolded optimization steps;
- 2) local trajectories remain sufficiently close to the broadcast global model to remain aggregation-compatible; and
- 3) the hyperparameter-generation policy is refined using post-aggregation validation performance.

Thus, the central problem is to jointly control client-side unfolded QNN optimization and server-side controller adaptation so that the global model achieves stable post-aggregation performance, reduced client drift, and improved client-level generalization under heterogeneous federated data.

B. QFL Setting and Notation

We consider a QFL system consisting of N distributed clients and one coordinating server. The clients collaboratively train a shared QNN model without exchanging raw local data. Let the client set be

$$\mathcal{C} = \{1, 2, \dots, N\}. \quad (1)$$

Training proceeds over T communication rounds indexed by

$$t \in \{0, 1, \dots, T-1\}. \quad (2)$$

At communication round t , the server maintains a global QNN parameter vector

$$\boldsymbol{\theta}^{(t)} \in \mathbb{R}^P, \quad (3)$$

where P denotes the number of trainable parameters in the variational quantum model. These parameters define a parameterized quantum circuit, denoted by

$$U(\mathbf{x}; \boldsymbol{\theta}^{(t)}), \quad (4)$$

where $\mathbf{x}$ is the classical input encoded into the quantum circuit and $\boldsymbol{\theta}^{(t)}$ parameterizes the trainable ansatz layers. The QNN output is obtained by measuring the resulting quantum state and applying classical post-processing to obtain prediction probabilities.

The server broadcasts $\boldsymbol{\theta}^{(t)}$ to the participating clients, and each client initializes its local QNN training from this global parameter vector.

Each client $i \in \mathcal{C}$ holds a private local dataset

$$\mathcal{D}_i = \{(\mathbf{x}_{i,j}, y_{i,j})\}_{j=1}^{n_i}, \quad (5)$$

where $n_i = |\mathcal{D}_i|$ is the number of local training samples. The client datasets may be statistically heterogeneous and non-identically distributed. Let

$$\mathcal{S}^{(t)} \subseteq \mathcal{C} \quad (6)$$

denote the subset of clients participating in round t . Unless otherwise stated, all subsequent expressions are written for participating clients $i \in \mathcal{S}^{(t)}$.

C. Local Quantum Model

Each participating client trains a parameterized quantum model represented as a QNN with trainable parameter vector $\boldsymbol{\theta}$. Given an input $\mathbf{x}$, the QNN applies a parameterized quantum circuit, followed by measurement and classical post-processing, to produce either a class-probability distribution or a predicted label. We denote the resulting client-side predictor by

$$f(\mathbf{x}; \boldsymbol{\theta}). \quad (7)$$

The empirical local objective at client i is defined as

$$\mathcal{L}_i(\boldsymbol{\theta}) = \frac{1}{n_i} \sum_{(\mathbf{x}, y) \in \mathcal{D}_i} \ell(f(\mathbf{x}; \boldsymbol{\theta}), y), \quad (8)$$

where $\ell(\cdot, \cdot)$ denotes the task-specific loss function, such as cross-entropy loss for classification.

In standard QFL, client i minimizes $\mathcal{L}_i(\boldsymbol{\theta})$ using a fixed local optimizer initialized from the broadcast global model $\boldsymbol{\theta}^{(t)}$. In contrast, DUQFL-Prox treats local QNN training as a structured unfolded optimization process. The local optimizer is decomposed into a finite sequence of adaptive SPSA-based update blocks, allowing the learning rate and perturbation scale to vary across clients, unfold steps, and communication rounds.

The proposed method is decomposed into three functional components. Algorithm 1 describes the client-side unfolded proximal QNN training procedure. Algorithm 2 describes one server-side communication round, including broadcast, local training, aggregation, and meta-loss evaluation. Algorithm 3 describes the outer SPSA-based refinement of the shared controller. This decomposition makes the bilevel structure explicit: the inner level optimizes local QNN parameters, while the outer level adapts the controller to improve post-aggregation global behaviour.

D. Deep-Unfolded Local Optimization

For each participating client $i \in \mathcal{S}^{(t)}$, the broadcast global model initializes the local unfolded trajectory,

$$\boldsymbol{\theta}_{i,0}^{(t)} \leftarrow \boldsymbol{\theta}^{(t)}. \quad (9)$$

Local training is then unfolded into K SPSA-based update blocks indexed by $k \in \{0, 1, \dots, K-1\}$. Each block updates the local QNN parameters according to

$$\boldsymbol{\theta}_{i,k+1}^{(t)} = \mathcal{U}_{\text{SPSA}} \left(\boldsymbol{\theta}_{i,k}^{(t)}, \eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}, \mathcal{L}_i^{\text{prox}} \right), \quad (10)$$

where $\eta_{i,k}^{(t)}$ and $\delta_{i,k}^{(t)}$ are the unfold-specific SPSA learning rate and perturbation scale generated by the controller.

Rather than always returning the last unfolded state, the client selects the validation-preferred checkpoint:

$$k_i^* = \arg \min_{k \in \{1, \dots, K\}} \mathcal{L}_{i,\text{val}} \left(\boldsymbol{\theta}_{i,k}^{(t)} \right), \quad (11)$$

and uploads

$$\boldsymbol{\theta}_{i,\star}^{(t)} = \boldsymbol{\theta}_{i,k_i^*}^{(t)}. \quad (12)$$

The intermediate unfolded states remain local to the client and are used only for trajectory construction, validation-based checkpoint selection, and optimization-summary generation.

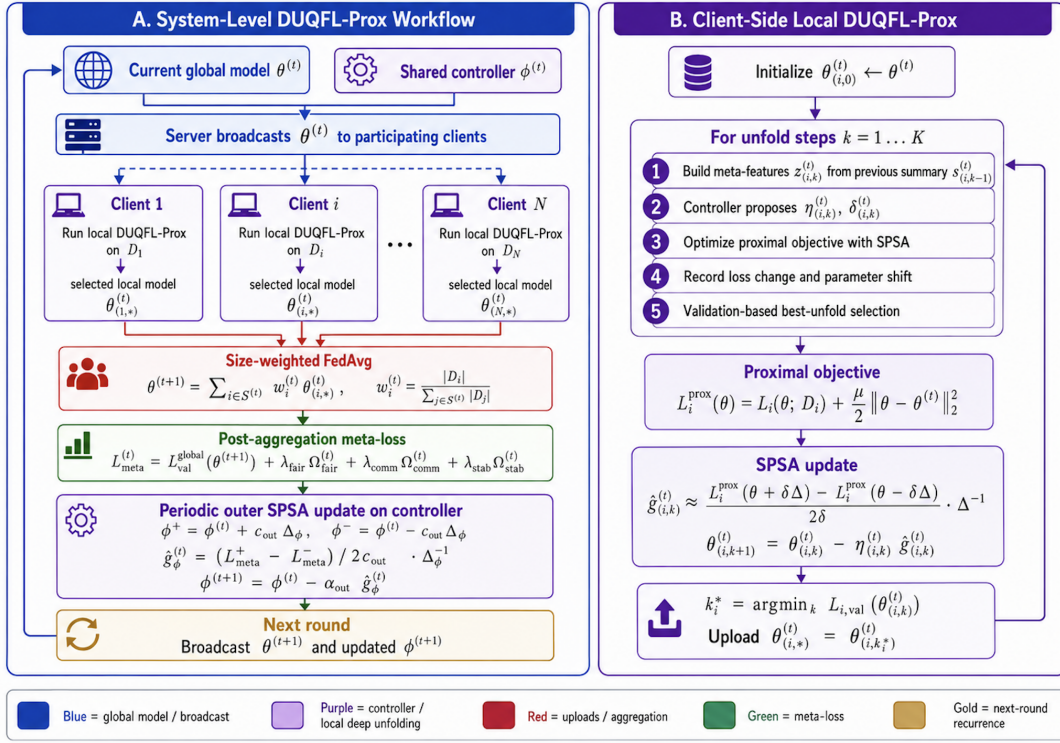


Fig. 2: Overview of DUQFL-Prox. The server broadcasts global QNN parameters to clients, which perform deep-unfolded SPSA optimization with controller-generated hyperparameters, proximal drift control, and best-unfold selection. The selected local models are aggregated at the server, while a meta-objective periodically updates the shared controller using post-aggregation validation performance.

E. Controller-Driven Hyperparameter Generation

The unfolded local optimizer requires step-specific SPSA hyperparameters for each client and unfold step. Instead of using a fixed learning rate and perturbation scale throughout local training, DUQFL-Prox uses a shared meta-controller to generate these quantities adaptively. The controller is parameterized by $\phi^{(t)}$ and is shared across participating clients at communication round t .

At unfold step k , client i constructs an optimization-state feature vector

$$\mathbf{z}_{i,k}^{(t)} \in \mathbb{R}^d, \quad (13)$$

which summarizes the current local optimization context. In our implementation, $\mathbf{z}_{i,k}^{(t)}$ includes normalized unfold progress, normalized communication-round progress, previous loss information, previous parameter displacement, client data fraction, and a client heterogeneity indicator.

The controller maps this feature vector to the SPSA learning rate and perturbation scale:

$$(\eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}) = \Gamma(\mathbf{z}_{i,k}^{(t)}; \phi^{(t)}), \quad (14)$$

where $\Gamma(\cdot)$ denotes the controller mapping.

In this work, $\Gamma(\cdot)$ is implemented as a clipped log-linear controller. Two parameter vectors, $\phi_{\eta}^{(t)}$ and $\phi_{\delta}^{(t)}$, generate the

logarithmic learning rate and perturbation scale:

$$\log \eta_{i,k}^{(t)} = \left(\phi_{\eta}^{(t)} \right)^{\top} \mathbf{z}_{i,k}^{(t)}, \quad (15)$$

$$\log \delta_{i,k}^{(t)} = \left(\phi_{\delta}^{(t)} \right)^{\top} \mathbf{z}_{i,k}^{(t)}. \quad (16)$$

The resulting values are exponentiated and clipped to predefined feasible intervals:

$$\eta_{i,k}^{(t)} = \text{clip} \left(\exp \left(\left(\phi_{\eta}^{(t)} \right)^{\top} \mathbf{z}_{i,k}^{(t)} \right), \eta_{\min}, \eta_{\max} \right), \quad (17)$$

$$\delta_{i,k}^{(t)} = \text{clip} \left(\exp \left(\left(\phi_{\delta}^{(t)} \right)^{\top} \mathbf{z}_{i,k}^{(t)} \right), \delta_{\min}, \delta_{\max} \right). \quad (18)$$

This controller provides the adaptive component of DUQFL-Prox by allowing the local SPSA behaviour to vary across clients, unfold steps, and communication rounds. The clipping bounds prevent numerically unstable hyperparameter values, while the proximal objective in Section III-F provides the stabilizing mechanism that restricts excessive client drift.

F. Proximal Drift Control

Client drift is a central challenge in federated learning, particularly under non-IID data distributions. In QFL, this issue is further amplified by stochastic SPSA updates, finite-shot measurement effects, and the non-convex loss landscape of variational quantum circuits. A client may reduce its local training loss while producing a parameter update that becomes

Algorithm 1 Client-Side Deep-Unfolded Proximal QNN Training

Require: Broadcast global parameters $\theta^{(t)}$, client dataset $\mathcal{D}_i$, client validation set $\mathcal{V}_i$, controller $\phi^{(t)}$, number of unfolds K , proximal coefficient μ

Ensure: Selected local model $\theta_{i,\star}^{(t)}$, optimization summaries $\{\mathbf{s}_{i,k}^{(t)}\}_{k=1}^K$

- 1: Initialize local parameters: $\theta_{i,0}^{(t)} \leftarrow \theta^{(t)}$
 - 2: Initialize optimization summary $\mathbf{s}_{i,0}^{(t)}$
 - 3: Initialize $L_{i,\star}^{(t)} \leftarrow \infty$ and $k_i^* \leftarrow 0$
 - 4: **for** $k = 0, 1, \dots, K-1$ **do**
 - 5: Construct optimization-state feature vector: $\mathbf{z}_{i,k}^{(t)} \leftarrow \Psi(k, t, \mathbf{s}_{i,k}^{(t)}, |\mathcal{D}_i|)$
 - 6: Generate unfold-specific SPSA hyperparameters: $\eta_{i,k}^{(t)}, \delta_{i,k}^{(t)} \leftarrow \Gamma(\mathbf{z}_{i,k}^{(t)}, \phi^{(t)})$
 - 7: Define the proximal local objective: $\mathcal{L}_i^{\text{prox}}(\theta) = \mathcal{L}_i(\theta) + \frac{\mu}{2} \|\theta - \theta^{(t)}\|_2^2$
 - 8: Compute pre-update proximal loss: $L_{i,k}^{\text{before}} \leftarrow \mathcal{L}_i^{\text{prox}}(\theta_{i,k}^{(t)})$
 - 9: Apply one SPSA-based proximal QNN update: $\theta_{i,k+1}^{(t)} \leftarrow \mathcal{U}_{\text{SPSA}}(\theta_{i,k}^{(t)}; \eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}, \mathcal{L}_i^{\text{prox}})$
 - 10: Compute post-update proximal loss: $L_{i,k}^{\text{after}} \leftarrow \mathcal{L}_i^{\text{prox}}(\theta_{i,k+1}^{(t)})$
 - 11: Compute unfold-step displacement: $\Delta\theta_{i,k}^{(t)} \leftarrow \theta_{i,k+1}^{(t)} - \theta_{i,k}^{(t)}$
 - 12: Evaluate validation loss: $L_{i,k}^{\text{val}} \leftarrow \mathcal{L}_{i,\text{val}}(\theta_{i,k+1}^{(t)}; \mathcal{V}_i)$
 - 13: **if** $L_{i,k}^{\text{val}} < L_{i,\star}^{(t)}$ **then**
 - 14: Update best unfolded state: $L_{i,\star}^{(t)} \leftarrow L_{i,k}^{\text{val}}, k_i^* \leftarrow k+1$
 - 15: **end if**
 - 16: Record optimization summary: $\mathbf{s}_{i,k+1}^{(t)} \leftarrow \text{Summarize}(L_{i,k}^{\text{before}}, L_{i,k}^{\text{after}}, L_{i,k}^{\text{val}}, \Delta\theta_{i,k}^{(t)})$
 - 17: **end for**
 - 18: Select best unfolded local model: $\theta_{i,\star}^{(t)} \leftarrow \theta_{i,k_i^*}^{(t)}$
 - 19: **return** $\theta_{i,\star}^{(t)}, \{\mathbf{s}_{i,k}^{(t)}\}_{k=1}^K$
-

overly specialized to its own local distribution and less compatible with global aggregation.

DUQFL-Prox addresses this issue by introducing a proximal penalty into the local client objective. Let $\theta^{(t)}$ denote the global model broadcast by the server at communication round t . For client i , the proximal local objective is defined as

$$\mathcal{L}_i^{\text{prox}}(\theta) = \mathcal{L}_i(\theta) + \frac{\mu}{2} \|\theta - \theta^{(t)}\|_2^2, \quad (19)$$

where $\mathcal{L}_i(\theta)$ is the empirical local QNN loss and $\mu \geq 0$ controls the strength of proximal regularization.

Within each unfolded local optimization step, SPSA minimizes $\mathcal{L}_i^{\text{prox}}$ rather than the unregularized local objective.

Thus, the proximal unfolded update is written as

$$\theta_{i,k+1}^{(t)} = \mathcal{U}_{\text{SPSA}}(\theta_{i,k}^{(t)}; \eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}, \mathcal{L}_i^{\text{prox}}). \quad (20)$$

The proximal term encourages local models to improve their client-specific objective while remaining close to the current global reference point. This is important in the unfolded setting because multiple adaptive local update blocks can improve flexibility, but may also increase the risk of excessive client movement. DUQFL-Prox therefore combines an adaptive component, provided by the controller-generated SPSA hyperparameters, with a stabilizing component, provided by proximal regularization.

To monitor local movement, we record the unfold-step displacement

$$\Delta\theta_{i,k}^{(t)} = \theta_{i,k+1}^{(t)} - \theta_{i,k}^{(t)}, \quad (21)$$

and its norm

$$\|\Delta\theta_{i,k}^{(t)}\|_2. \quad (22)$$

This quantity is used as a diagnostic indicator of local client movement and can also be included in the controller state features for subsequent unfold steps. Overall, the proximal formulation helps produce adaptive yet aggregation-compatible local QNN updates under heterogeneous federated data.

G. Best-Unfold Model Selection

The last unfolded state is not necessarily the best local model to upload. Later unfold steps may continue to reduce local training loss while degrading validation behaviour or increasing local specialization. Therefore, DUQFL-Prox uses validation-based checkpoint selection over the local unfolded trajectory.

After generating the candidate states

$$\{\theta_{i,k}^{(t)}\}_{k=1}^K, \quad (23)$$

client i selects the unfold index with the lowest validation loss:

$$k_i^* = \arg \min_{k \in \{1, \dots, K\}} \mathcal{L}_{i,\text{val}}(\theta_{i,k}^{(t)}). \quad (24)$$

The selected local model is then

$$\theta_{i,\star}^{(t)} = \theta_{i,k_i^*}^{(t)}. \quad (25)$$

Only $\theta_{i,\star}^{(t)}$ is uploaded to the server. The remaining unfolded states are retained locally and are used only for trajectory construction, checkpoint selection, and optimization-summary generation. This selection mechanism prevents over-aggressive later unfold steps from dominating the uploaded client update.

H. Server Aggregation

After receiving the selected unfolded local models from participating clients, the server constructs the next global model using sample-size-weighted FedAvg:

$$\theta^{(t+1)} = \sum_{i \in \mathcal{S}^{(t)}} w_i^{(t)} \theta_{i,\star}^{(t)}, \quad (26)$$

Algorithm 2 One Federated Round of DUQFL-Prox

Require: Client set $\mathcal{C}$, current global model $\theta^{(t)}$, current controller $\phi^{(t)}$, participating clients $\mathcal{S}^{(t)}$, number of unfolds K , proximal coefficient μ

Ensure: Updated global model $\theta^{(t+1)}$, outer meta-loss $\mathcal{L}_{\text{meta}}^{(t)}$

- 1: Server broadcasts $\theta^{(t)}$ to all clients in $\mathcal{S}^{(t)}$
- 2: **for all** clients $i \in \mathcal{S}^{(t)}$ **in parallel do**
- 3: Execute Algorithm 1: $\left(\theta_{i,\star}^{(t)}, \{\mathbf{s}_{i,k}^{(t)}\}_{k=1}^K\right) \leftarrow \text{LocalDUQFLProx}\left(\theta^{(t)}, \mathcal{D}_i, \mathcal{V}_i, \phi^{(t)}, K, \mu\right)$
- 4: **end for**
- 5: Compute sample-size aggregation weights: $w_i^{(t)} = \frac{|\mathcal{D}_i|}{\sum_{j \in \mathcal{S}^{(t)}} |\mathcal{D}_j|}$
- 6: Aggregate selected unfolded local models: $\theta^{(t+1)} \leftarrow \sum_{i \in \mathcal{S}^{(t)}} w_i^{(t)} \theta_{i,\star}^{(t)}$
- 7: Evaluate post-aggregation meta-loss: $\mathcal{L}_{\text{meta}}^{(t)} \leftarrow \mathcal{L}_{\text{val}}^{\text{global}}\left(\theta^{(t+1)}\right) + \lambda_{\text{fair}} \Omega_{\text{fair}}^{(t)} + \lambda_{\text{comm}} \Omega_{\text{comm}}^{(t)} + \lambda_{\text{stab}} \Omega_{\text{stab}}^{(t)}$
- 8: **return** $\theta^{(t+1)}, \mathcal{L}_{\text{meta}}^{(t)}$

Algorithm 3 Outer SPSA-Based Controller Update

Require: Current controller $\phi^{(t)}$, current global model $\theta^{(t)}$, participating clients $\mathcal{S}^{(t)}$, outer learning rate α_{out} , outer perturbation radius c_{out}

Ensure: Updated controller $\phi^{(t+1)}$

- 1: Sample Rademacher perturbation vector Δ_ϕ
- 2: Construct perturbed controllers: $\phi^+ = \phi^{(t)} + c_{\text{out}} \Delta_\phi$, $\phi^- = \phi^{(t)} - c_{\text{out}} \Delta_\phi$
- 3: Evaluate one federated round under ϕ^+ using Algorithm 2 to obtain: $\mathcal{L}_{\text{meta}}^+$
- 4: Evaluate one federated round under ϕ^- using Algorithm 2 to obtain: $\mathcal{L}_{\text{meta}}^-$
- 5: Estimate the outer SPSA gradient: $\hat{\mathbf{g}}_\phi^{(t)} \leftarrow \frac{\mathcal{L}_{\text{meta}}^+ - \mathcal{L}_{\text{meta}}^-}{2c_{\text{out}}} \Delta_\phi^{-1}$
- 6: Update the controller: $\phi^{(t+1)} \leftarrow \phi^{(t)} - \alpha_{\text{out}} \hat{\mathbf{g}}_\phi^{(t)}$
- 7: **return** $\phi^{(t+1)}$

where $w_i^{(t)}$ denotes the aggregation weight of client i . The weights satisfy

$$w_i^{(t)} \geq 0, \quad \sum_{i \in \mathcal{S}^{(t)}} w_i^{(t)} = 1. \quad (27)$$

In this work, the sample-size-weighted coefficient is

$$w_i^{(t)} = \frac{n_i}{\sum_{j \in \mathcal{S}^{(t)}} n_j}, \quad (28)$$

where $n_i = |\mathcal{D}_i|$ is the number of local training samples available to client i in the current round.

Thus, the server does not aggregate every unfolded state. It aggregates only the validation-selected local checkpoint $\theta_{i,\star}^{(t)}$ from each participating client.

I. Outer Meta-Objective and Controller Update

The controller is not treated as a fixed hyperparameter generator. Instead, it is periodically updated using an outer

meta-objective evaluated after server aggregation. Let $\mathcal{D}_{\text{val}}$ denote the validation data used to measure post-aggregation global performance. At communication round t , the outer meta-loss is defined as

$$\mathcal{L}_{\text{meta}}^{(t)} = \mathcal{L}_{\text{val}}^{\text{global}}\left(\theta^{(t+1)}\right) + \lambda_{\text{fair}} \Omega_{\text{fair}}^{(t)} + \lambda_{\text{comm}} \Omega_{\text{comm}}^{(t)} + \lambda_{\text{stab}} \Omega_{\text{stab}}^{(t)}. \quad (29)$$

Here, $\mathcal{L}_{\text{val}}^{\text{global}}$ is the validation loss of the aggregated global model, $\Omega_{\text{fair}}^{(t)}$ measures client-level performance imbalance, $\Omega_{\text{comm}}^{(t)}$ accounts for communication cost, and $\Omega_{\text{stab}}^{(t)}$ penalizes unstable optimization trajectories. The coefficients λ_{fair} , λ_{comm} , and λ_{stab} control the relative contribution of these terms.

To update the controller, DUQFL-Prox applies an outer SPSA step to $\phi^{(t)}$. A Rademacher perturbation vector Δ_ϕ is sampled, and two perturbed controllers are formed:

$$\phi^+ = \phi^{(t)} + c_{\text{out}} \Delta_\phi, \quad \phi^- = \phi^{(t)} - c_{\text{out}} \Delta_\phi. \quad (30)$$

The corresponding perturbed meta-losses, $\mathcal{L}_{\text{meta}}^+$ and $\mathcal{L}_{\text{meta}}^-$, are obtained by evaluating virtual federated rounds under ϕ^+ and ϕ^- , respectively. The outer SPSA gradient estimate is then

$$\hat{\mathbf{g}}_\phi^{(t)} = \frac{\mathcal{L}_{\text{meta}}^+ - \mathcal{L}_{\text{meta}}^-}{2c_{\text{out}}} \Delta_\phi^{-1}. \quad (31)$$

Since each perturbation entry satisfies $\Delta_{\phi,j} \in \{-1, +1\}$, we have $\Delta_{\phi,j}^{-1} = \Delta_{\phi,j}$.

The controller is updated as

$$\phi^{(t+1)} = \phi^{(t)} - \alpha_{\text{out}} \hat{\mathbf{g}}_\phi^{(t)}, \quad (32)$$

where α_{out} is the outer learning rate.

This establishes a bilevel learning structure. The inner level performs client-side deep-unfolded proximal SPSA optimization of QNN parameters, while the outer level adapts the shared controller so that future local optimization trajectories become more compatible with post-aggregation global performance. The outer update is applied periodically rather than necessarily at every round, which reduces computational overhead while still allowing the controller to improve across training.

IV. EXPERIMENTAL SETUP

This section describes the implementation environment, quantum model configuration, datasets, federated partitioning strategy, baseline methods, evaluation metrics, and reproducibility protocol used to evaluate DUQFL-Prox. The objective of the experiments is to assess not only final global accuracy, but also client-level generalization, train-test gap, client fairness, and stability under heterogeneous federated data.

A. Implementation Details

All experiments were implemented in Python using Visual Studio Code as the main development environment. Quantum models were implemented using Qiskit and Qiskit Machine Learning, with Qiskit Aer used for simulator-based experiments. The main Python libraries used were NumPy, pandas, scikit-learn, matplotlib, seaborn, Qiskit, Qiskit Aer, and Qiskit

Machine Learning. Fixed random seeds were used for dataset splitting, client partitioning, QNN parameter initialization, and optimizer-related stochasticity wherever supported.

The main experimental configuration is summarized in Table II. The reported values correspond to the default configuration used in the main experiments; dataset-specific adjustments were made when necessary due to dataset capacity or computational constraints.

TABLE II: Main experimental configuration used in DUQFL-Prox experiments.

Setting	Value
Programming language	Python 3.11.14
Development environment	Visual Studio Code
Quantum framework	Qiskit, Qiskit Machine Learning
Simulator	Qiskit Aer simulator
Feature map	ZZFeatureMap
Ansatz	RealAmplitudes
Optimizer	SPSA / DUQFL-Prox SPSA
Number of clients	5 or 10
Communication rounds	10–50 depending on dataset capacity
Unfold steps	$K = 5$
SPSA iterations per unfold	5–10
Initial learning rate	0.13
Initial perturbation	0.13
Proximal coefficient	$\mu = 10^{-2}$
Shots	1024

B. Quantum Neural Network Architecture

Each client model was implemented as a parameterized QNN. The classical input features were first preprocessed into a low-dimensional representation and then encoded into a quantum circuit using a ZZFeatureMap. A RealAmplitudes ansatz was used as the trainable variational circuit. The number of qubits was determined by the number of retained input features after preprocessing. In the main experiments, we used either two or four QNN input features depending on the dataset and computational budget.

Let $\mathbf{x} \in \mathbb{R}^d$ denote the preprocessed input vector. The QNN maps $\mathbf{x}$ to a quantum state through the feature map and then applies a trainable ansatz parameterized by θ . Measurement outcomes are classically post-processed to obtain class probabilities or predicted labels. All compared methods used the same QNN architecture for a given dataset to ensure a fair comparison.

C. Datasets and Preprocessing

The proposed DUQFL-Prox framework was evaluated on datasets from genomics and financial fraud detection. Since current QNN models are constrained by the number of available qubits and circuit depth, each dataset was transformed into a low-dimensional QNN-compatible representation before federated training. Table III summarizes the dataset usage, preprocessing pipeline, and final QNN input dimension. The BAF dataset is taken from the Bank Account Fraud Dataset Suite introduced by Jesus et al. [38]. The dataset suite was designed as a privacy-preserving, large-scale tabular benchmark for bank account-opening fraud detection, with realistic challenges including temporal dynamics, severe class

TABLE III: Datasets used for evaluating DUQFL-Prox across multiple domains.

Dataset	Domain	Task	Source	QNN features
BAF	Finance	Bank fraud detection	[38]	4
Genome	Genomics	Genomic sequencing	[39]	2 or 4

imbalance, and bias/fairness-related distributional shifts. The Genome experiments use the DemoHumanOrWorm task from the Genomic Benchmarks suite [39]. Genomic Benchmarks provides curated datasets for genomic sequence classification and offers standardized access through common machine-learning and deep-learning interfaces.

1) *BAF Dataset*: For the BAF, Bank Account fraud-detection experiment, the target variable was `fraud_bool`. After removing missing values, the dataset was optionally subsampled using stratified sampling to preserve the class distribution under the QNN computational budget. The data were split into training, validation, and test sets before feature encoding and scaling to avoid data leakage. Categorical variables were encoded using ordinal encoding with support for unseen categories, while numerical variables were standardized. PCA was then applied to obtain a low-dimensional representation compatible with the number of QNN qubits. Finally, the PCA features were optionally scaled to the quantum angle range $[0, \pi]$. In the reported BAF run, the resulting split contained 2,999 training samples, 750 validation samples, and 1,250 test samples, using four QNN features/qubits.

2) *Genome Dataset*: For the Genome experiment, we used the DemoHumanOrWorm task from the Genomic Benchmarks suite. Each DNA sequence was converted into a numerical vector using a fixed word-size encoding strategy. Specifically, all unique words of length `word_size` were assigned integer identifiers, and each sequence was represented by the corresponding sequence of integer word indices. The resulting vectors were shuffled and scaled using MinMax scaling to obtain bounded QNN-compatible input features. In the implementation used in this work, 10,000 processed records were used for training and 2,000 records were used for testing.

The BAF and Genome datasets are used as representative intelligent-service tasks because they capture two unique privacy-sensitive distributed decision setting challenges: *financial fraud detection and genomic classification*.

V. EXPERIMENTAL RESULTS AND ANALYSIS

We evaluate DUQFL-Prox against representative QFL baselines and ablation variants. The analysis is organized around four questions: (i) whether DUQFL-Prox improves global and client-level performance, (ii) whether the proposed method reduces train–test generalization gap and client fairness gap, (iii) how DUQFL-Prox behaves under severe class imbalance, and (iv) whether the learned global QNN checkpoints remain executable on real IBM quantum hardware.

We report both global and client-level metrics because global accuracy alone is insufficient to evaluate heterogeneous QFL. A method may obtain strong global accuracy while still producing unstable or imbalanced performance across clients. Therefore, in addition to global accuracy, we analyse

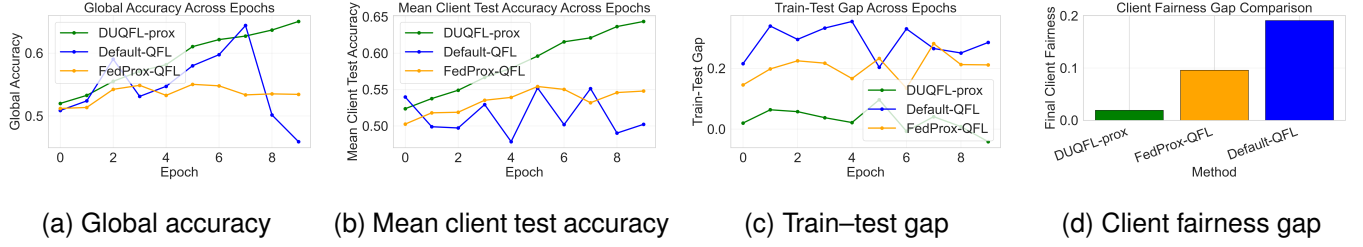


Fig. 3: Federated performance comparison on the BAF dataset. DUQFL-Prox improves final global accuracy and mean client test accuracy while substantially reducing the train–test gap and client fairness gap. Lower values are preferable for train–test gap and fairness gap.

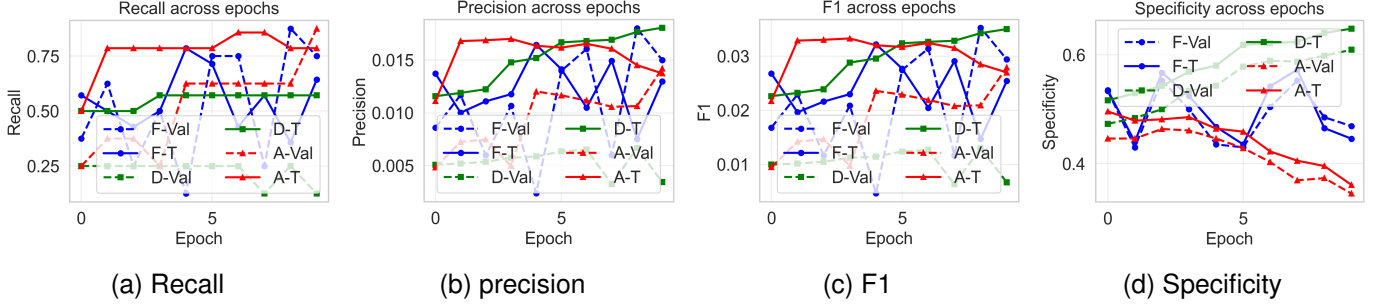


Fig. 4: Epoch-wise ROC-AUC, PR-AUC, MCC, and specificity trajectories on the BAF dataset. DUQFL-Prox shows more stable late-epoch behaviour on test ROC-AUC, PR-AUC, and MCC, while Adam-QFL exhibits a recall-heavy operating regime with declining specificity.

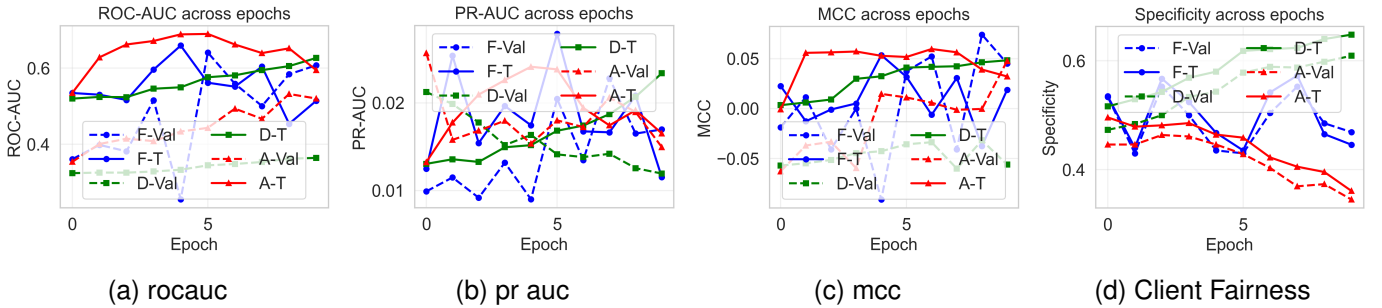


Fig. 5: BAF Data

mean client test accuracy, train–test gap, client fairness gap, precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, and specificity.

A. Results on the BAF Dataset

The BAF dataset represents a highly imbalanced financial fraud-detection task. This setting is challenging for QFL because the minority class is rare and the client partitions are non-IID. Consequently, high global accuracy alone does not necessarily imply good fraud-detection behaviour. We therefore evaluate BAF using both federated learning metrics and imbalance-aware classification metrics.

Figure 3 compares DUQFL-Prox with Default-QFL and FedProx-QFL using global accuracy, mean client test accuracy, train–test gap, and client fairness gap. DUQFL-Prox achieves the strongest final global accuracy, reaching approximately 0.6504, compared with approximately 0.5344 for FedProx-QFL and 0.4592 for Default-QFL. Although Default-QFL

reaches a temporary peak during intermediate rounds, its late-round performance decreases substantially, indicating unstable post-aggregation behaviour under the non-IID BAF setting.

The client-level results provide a clearer indication of the benefit of DUQFL-Prox. The proposed method obtains the highest final mean client test accuracy, approximately 0.6436, while FedProx-QFL and Default-QFL obtain approximately 0.5480 and 0.5022, respectively. This suggests that DUQFL-Prox improves not only the aggregated global model, but also the generalization behaviour observed across distributed clients.

The train–test gap further supports this conclusion. Default-QFL and FedProx-QFL show relatively large final train–test gaps, approximately 0.2854 and 0.2116, respectively. In contrast, DUQFL-Prox obtains a near-zero final gap. Since lower train–test gap is preferable, this indicates that DUQFL-Prox reduces local over-specialization and improves generalization under heterogeneous client data.

The client fairness gap, measured as the $P90 - P10$ spread of client test accuracies, is also lowest for DUQFL-Prox. The final fairness gap is approximately 0.0193 for DUQFL-Prox, compared with 0.0958 for FedProx-QFL and 0.1910 for Default-QFL. This result is important because a high global accuracy can hide poor performance on difficult or minority clients. The low fairness gap shows that DUQFL-Prox produces more balanced client-level performance.

B. Imbalance-Aware Classification Behaviour on BAF

Because the BAF dataset is severely class-imbalanced, we further analyse precision, recall, F1-score, specificity, ROC-AUC, PR-AUC, and MCC on validation and test splits. In Figures 4 and 5, the prefixes F, D, and A denote FedAvg-default, DUQFL-Prox, and FedAvg-tuned-Adam, respectively, while Val and T denote validation and test splits.

The results reveal different operating behaviours. FedAvg-tuned-Adam achieves high recall for much of training, indicating aggressive minority-class detection. However, this behaviour is accompanied by a decline in specificity, which suggests a larger number of false positives. Thus, Adam-QFL behaves as a recall-oriented baseline but provides a less selective operating point.

DUQFL-Prox shows a more balanced late-epoch behaviour. It provides more consistent improvements in test F1, MCC, ROC-AUC, precision, and specificity. This suggests that DUQFL-Prox does not simply increase minority-class detection at the expense of false positives; instead, it maintains a more balanced recall-specificity trade-off. FedAvg-default exhibits stronger fluctuations across epochs, particularly for MCC, F1-score, and ROC-AUC, indicating weaker stability under severe class imbalance.

The validation curves are noisier than the test curves, especially for PR-AUC, MCC, precision, and recall. This is expected because the validation split contains very few positive fraud samples. Therefore, epoch-wise trajectories are important in addition to checkpoint-based summaries, since they reveal the underlying training dynamics and the recall-specificity trade-off among the methods.

C. Results on the Genome Dataset

The Genome dataset presents a more nuanced comparison. As shown in Figure 6, FedProx-QFL achieves the highest final global accuracy, reaching approximately 0.85. DUQFL-Prox remains competitive, with final global accuracy around 0.82–0.83, while Default-QFL ends with substantially lower global accuracy. This result shows that DUQFL-Prox is not always the single best method in terms of final global accuracy.

However, the client-level metrics show a different and important trend. DUQFL-Prox achieves the highest final mean client test accuracy, approximately 0.80, outperforming the other compared methods. This indicates that DUQFL-Prox provides stronger generalization across distributed clients, even when another baseline obtains slightly higher final global accuracy.

The train–test gap and client fairness gap further support this interpretation. DUQFL-Prox obtains the lowest final train–test

gap, close to zero, suggesting reduced overfitting and improved client-level generalization. It also achieves the lowest final client fairness gap, indicating that its performance is more balanced across clients. DUQFL-best and DUQFL-drift also improve over Default-QFL, but DUQFL-Prox provides the strongest overall stability and fairness profile.

Therefore, the Genome experiment supports a more federated-learning-relevant conclusion: DUQFL-Prox is not merely an accuracy-maximizing method, but a stability-aware and generalization-aware QFL method. It provides the best trade-off among global accuracy, mean client test accuracy, train–test gap, and fairness under heterogeneous client data.

D. Cross-Dataset Comparison

Table IV summarizes the qualitative behaviour of the compared methods across the Genome and BAF datasets. The results show that DUQFL-Prox provides the most consistent stability–generalization trade-off across datasets.

On BAF, DUQFL-Prox achieves the strongest performance across final global accuracy, mean client test accuracy, train–test gap, and client fairness gap. This indicates that proximal deep-unfolded QNN optimization is particularly effective under severe class imbalance and non-IID client partitions.

On Genome, FedProx-QFL achieves the highest final global accuracy, whereas DUQFL-Prox achieves the strongest client-level generalization, lowest train–test gap, and lowest fairness gap. This distinction is important because a single global test metric does not fully characterize federated performance. The Genome results show that DUQFL-Prox improves the reliability and balance of client-level performance even when another method slightly improves the final centralized global accuracy.

Overall, the cross-dataset results indicate that DUQFL-Prox should be interpreted as a stability-aware and generalization-aware QFL method. Its advantage lies in improving the quality and aggregation compatibility of local QNN updates through deep-unfolded optimization, validation-based checkpoint selection, and proximal drift control.

E. Controller Behaviour and Hyperparameter Adaptation

Figure 8 analyses the controller trace logs of DUQFL-Prox. The generated learning rate η and perturbation scale δ remain within stable numerical ranges while varying across the unfolded training process. The local loss-reduction curve shows that the unfolded SPSA blocks produce measurable improvement in the client objective, particularly in the early unfold steps. The selected-checkpoint distribution further shows that the last unfolded state is not always uploaded, supporting the use of validation-based best-unfold selection. Together, these results provide empirical evidence that DUQFL-Prox uses controller-guided adaptive optimization rather than a fixed local SPSA schedule.

F. Real IBM Quantum Hardware Validation

To examine practical deployability, we evaluated selected trained global QNN checkpoints on IBM Quantum hardware.

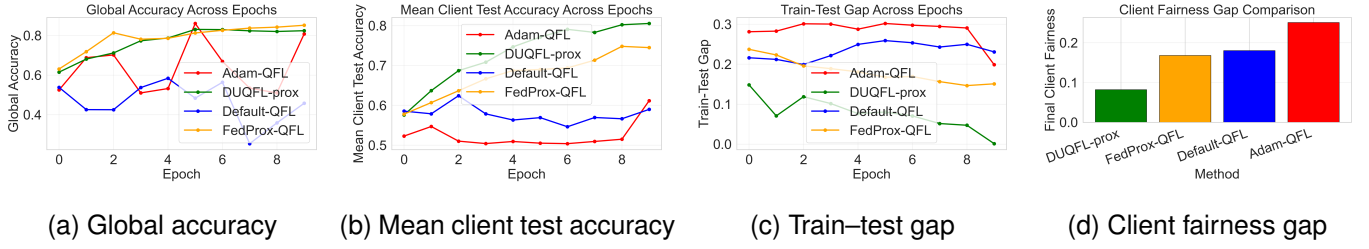


Fig. 6: Federated performance comparison on the Genome dataset. FedProx-QFL achieves the strongest final global accuracy, while DUQFL-Prox provides the best client-level generalization, lowest train-test gap, and lowest fairness gap.

TABLE IV: Qualitative comparison of QFL and DUQFL methods across Genome and BAF datasets. Here, $\checkmark\checkmark$ denotes the strongest performance, $\checkmark$ denotes competitive or moderate performance, and $-$ denotes weak, unstable, or less consistent performance. For train-test gap, client drift/regularization, and fairness/balance, lower values are preferable.

Dataset	Method	Final global accuracy	Peak global accuracy	Mean client test accuracy	Low train-test gap	Low client drift / regularization	Client fairness / balance
Genome	Adam-QFL	$-$	$\checkmark\checkmark$	$-$	$-$	$-$	$-$
	DUQFL-Prox	$\checkmark$	$-$	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$
	Default-QFL	$-$	$-$	$\checkmark$	$-$	$\checkmark$	$\checkmark$
	FedProx-QFL	$\checkmark\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
BAF	Adam-QFL	$-$	$-$	$-$	$-$	$-$	$-$
	DUQFL-Prox	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$	$\checkmark\checkmark$
	Default-QFL	$-$	$\checkmark$	$\checkmark$	$-$	$\checkmark$	$\checkmark$
	FedProx-QFL	$\checkmark$	$-$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

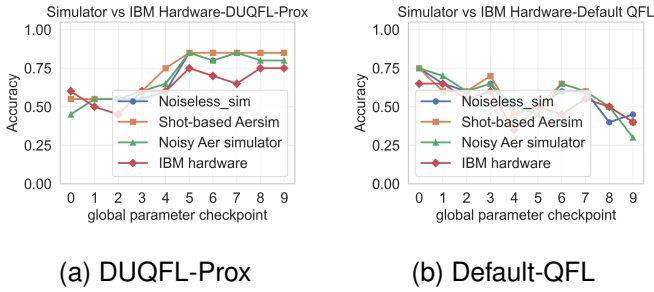


Fig. 7: Simulator and real IBM quantum hardware validation of trained global QFL checkpoints. DUQFL-Prox shows a more stable hardware trajectory than Default-QFL across saved global parameter checkpoints.

Due to the high queue time and execution cost of repeatedly training the full federated process on real quantum devices, hardware execution was used for post-training validation rather than full hardware-in-the-loop federated training. Specifically, saved global parameter checkpoints were assigned to the trained QNN circuit, transpiled for the selected IBM backend, and executed using finite-shot measurement. The resulting hardware accuracy was compared with noiseless simulator, shot-based Aer simulator, and backend-inspired noisy Aer simulator results.

Figure 7 compares DUQFL-Prox and Default-QFL across saved global checkpoints. DUQFL-Prox shows a more stable and improving hardware trajectory than Default-QFL and remains reasonably aligned with the simulator curves in later checkpoints. The difference between simulator and hardware results is expected due to finite-shot uncertainty, device noise, calibration drift, transpilation constraints, and backend-specific routing overhead.

These results should not be interpreted as full federated

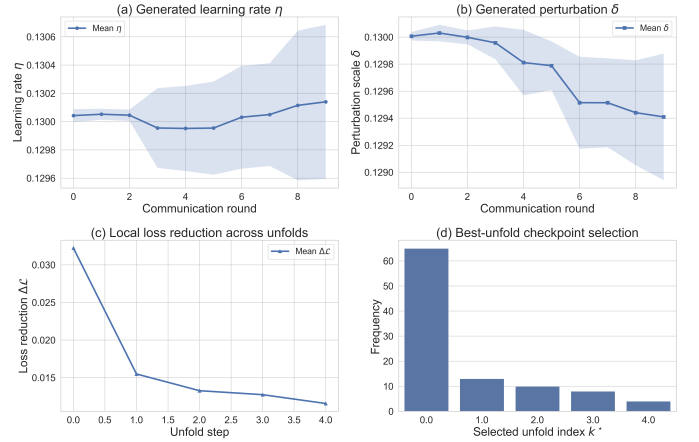


Fig. 8: Controller behaviour and hyperparameter adaptation in DUQFL-Prox on the BAF dataset. The trace logs show the controller-generated SPSA learning rate η , perturbation scale δ , local loss reduction $\Delta\mathcal{L}$, and selected best-unfold checkpoint k^* . These results verify that DUQFL-Prox performs adaptive local QNN optimization rather than using a fixed SPSA schedule.

ated training on IBM hardware. Rather, they provide post-training hardware feasibility evidence, showing that the learned DUQFL-Prox global checkpoints remain executable on real quantum hardware and retain a more stable trajectory than the Default-QFL baseline.

VI. CONCLUSION

The proposed DUQFL-Prox framework is supported by three theoretical observations. First, the optional step-projection mechanism bounds each unfolded local update and therefore bounds cumulative client drift across the unfolded trajectory. Second, the proximal regularization term acts

as a drift-control mechanism by penalizing deviation from the broadcast global model, balancing local adaptation with aggregation compatibility. Third, under standard smoothness assumptions on the post-aggregation meta-objective, the outer SPSA update provides a zeroth-order controller-update direction that supports descent-oriented adaptation in expectation. Our results support the use of DUQFL-Prox as a drift-stable QFL framework for intelligent services that require privacy preservation, fairness, and reliable generalization across heterogeneous clients. These results do not constitute a full convergence guaranty for arbitrary nonconvex QFL systems; instead, they characterize the stability, drift-control, and meta-optimization behavior induced by DUQFL-Prox. Detailed statements and proofs are provided in Appendix A.

REFERENCES

- [1] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.
- [2] K. Yu, X. Zhang, Z. Ye, G.-D. Guo, and S. Lin, "Quantum federated learning based on gradient descent," *arXiv preprint arXiv:2212.12913*, 2022.
- [3] R. Huang, X. Tan, and Q. Xu, "Quantum federated learning with decentralized data," *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 28, no. 4, pp. 1–10, 2022.
- [4] D. C. Nguyen, M. R. Uddin, S. Shaon, R. Rahman, O. Dobre, and D. Niyato, "Quantum federated learning: A comprehensive survey," *arXiv preprint arXiv:2508.15998*, 2025.
- [5] S. Jerbi, C. Gyurik, S. C. Marshall, R. Molteni, and V. Dunjko, "Shadows of quantum machine learning," *Nature Communications*, vol. 15, no. 1, p. 5676, 2024.
- [6] J. Xu *et al.*, "A holistic and hybrid service selection strategy for mecbased uav last-mile delivery systems," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3022–3036, 2024.
- [7] A. Hamdi *et al.*, "Drone-as-a-service: Research challenges and directions," *Proceedings of the IEEE*, vol. 113, no. 5, pp. 416–442, 2025.
- [8] A. G. Neiat, A. Bouguettaya, and M. Bahutair, "A deep reinforcement learning approach for composing moving iot services," *IEEE Transactions on Services Computing*, vol. 15, no. 5, pp. 2538–2550, 2022.
- [9] P. Kairouz and H. B. McMahan, "Advances and open problems in federated learning," *Foundations and trends in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [10] A. Reiszadeh, I. Tziotis, H. Hassani, A. Mokhtari, and R. Pedarsani, "Straggler-resilient federated learning: Leveraging the interplay between statistical accuracy and system heterogeneity," *IEEE Journal on Selected Areas in Information Theory*, vol. 3, no. 2, pp. 197–205, 2022.
- [11] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, 2021, pp. 19–35.
- [12] Y. J. Cho, J. Wang, and G. Joshi, "Towards understanding biased client selection in federated learning," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2022, pp. 10351–10375.
- [13] K. Selialia, Y. Chandio, and F. M. Anwar, "Mitigating group bias in federated learning for heterogeneous devices," in *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*, 2024, pp. 1043–1054.
- [14] J. C. Spall, "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation," *IEEE transactions on automatic control*, vol. 37, no. 3, pp. 332–341, 2002.
- [15] IBM Quantum, "SPSA Optimizer," https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit_algorithms.optimizers.SPSA, accessed: 2026-05-17.
- [16] W. Li, S. Lu, and D.-L. Deng, "Quantum federated learning through blind quantum computing," *Science China Physics, Mechanics & Astronomy*, vol. 64, no. 10, p. 100312, 2021.
- [17] C. Chu, L. Jiang, and F. Chen, "Cryptoqfl: Quantum federated learning on encrypted data," in *Proc. IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1, 2023, pp. 1231–1237.
- [18] W. J. Yun, J. P. Kim, H. Baek, S. Jung, J. Park, M. Bennis, and J. Kim, "Quantum federated learning with entanglement controlled circuits and superposition coding," *arXiv preprint arXiv:2212.01732*, 2022.
- [19] Y. F. Hanna, A. A. Khater, M. El-Bardini, and A. M. El-Nagar, "Real time adaptive pid controller based on quantum neural network for nonlinear systems," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106952, 2023.
- [20] N. Innan, A. Marchisio, M. Bennai, and M. Shafique, "Qfnn-ffd: Quantum federated neural network for financial fraud detection," in *2025 IEEE International Conference on Quantum Software (QSW)*. IEEE, 2025, pp. 41–47.
- [21] S. Majumder, S. Ray, M. Dasgupta, P. Bhattacharya, T. R. Gadekallu, and G. Srivastava, "Quantum state verification scheme for fraud detection in iot-assisted quantum-blockchain networks," *IEEE Transactions on Services Computing*, vol. 19, no. 1, pp. 616–627, 2026.
- [22] X. Zhang, J. Zhao, C. Xu, H. Wang, and Y. Zhang, "Dopiv: Post-quantum secure identity-based data outsourcing with public integrity verification in cloud storage," *IEEE Transactions on Services Computing*, vol. 15, no. 1, pp. 334–345, 2022.
- [23] W. Yamany, N. Moustafa, and B. Turnbull, "Oqfl: An optimized quantum-based federated learning framework for defending against adversarial attacks in intelligent transportation systems," *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [24] B. Narottama and S. Y. Shin, "Federated quantum neural network with quantum teleportation for resource optimization in future wireless communication," *IEEE Transactions on Vehicular Technology*, 2023.
- [25] S. R. Pokhrel, N. Yash, J. Kua, G. Li, and L. Pan, "Quantum federated learning experiments in the cloud with data encoding," *arXiv preprint arXiv:2405.00909*, 2024.
- [26] H. Zhao, "Non-iid quantum federated learning with one-shot communication complexity," *Quantum Machine Intelligence*, vol. 5, no. 1, p. 3, 2023.
- [27] J. Qi, X.-L. Zhang, and J. Tejedor, "Optimizing quantum federated learning based on federated quantum natural gradient descent," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [28] M. Chehimi, S. Y.-C. Chen, W. Saad, D. Towsley, and M. Debbah, "Foundations of quantum federated learning over classical and quantum networks," *IEEE Network*, 2023.
- [29] Q. Xia and Q. Li, "Quantumfed: A federated learning framework for collaborative quantum training," in *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2021, pp. 1–6.
- [30] D. Ning, Y.-F. Ge, E. Bertino, Z. Zheng, Y. Jiang, and H. Wang, "Adpfl: A privacy-preserving federated learning framework through adaptive model pruning on non-iid data," *IEEE Transactions on Services Computing*, pp. 1–15, 2026.
- [31] S. I. Nanayakkara and S. R. Pokhrel, "New insights on unfolding and fine-tuning quantum federated learning," *arXiv preprint arXiv:2506.20016*, 2025.
- [32] A. Nakai-Kasai and T. Wadayama, "Deep unfolding-based weighted averaging for federated learning under device and statistical heterogeneous environments," *IEICE Transactions on Communications*, vol. 108, no. 4, pp. 411–420, 2024.
- [33] Qiskit Community, "Spsa - qiskit algorithms documentation," https://qiskit-community.github.io/qiskit-algorithms/stubs/qiskit_algorithms.optimizers.SPSA.html, 2025, accessed: 2026-05-10.
- [34] J. C. Spall, "An overview of the simultaneous perturbation method for efficient optimization," *Johns Hopkins apl technical digest*, vol. 19, no. 4, pp. 482–492, 1998.
- [35] N. Shlezinger, S. Segarra, Y. Zhang, D. Avrahami, Z. Davidov, T. Routtenberg, and Y. C. Eldar, "Deep unfolding: Recent developments, theory, and design guidelines," *arXiv preprint arXiv:2512.03768*, 2025.
- [36] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "Scaffold: Stochastic controlled averaging for federated learning," pp. 5132–5143, 2020.
- [37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [38] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. Ribeiro, J. Gama, and P. Bizarro, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ml evaluation," *Advances in Neural Information Processing Systems*, vol. 35, pp. 33563–33575, 2022.
- [39] K. Grešová, V. Martinek, D. Čechák, P. Šimeček, and P. Alexiou, "Genomic benchmarks: a collection of datasets for genomic sequence classification," *BMC Genomic Data*, vol. 24, no. 1, p. 25, 2023.

TABLE V: Summary of notation used in DUQFL-Prox.

Notation	Description
$\mathcal{C} = \{1, \dots, N\}$	Set of all clients
N	Number of clients
$t \in \{0, \dots, T-1\}$	Communication round index
T	Number of communication rounds
$\mathcal{S}^{(t)} \subseteq \mathcal{C}$	Participating client set at round t
$\mathcal{D}_i$	Local training dataset of client i
$n_i = \mathcal{D}_i $	Number of local training samples at client i
$\mathcal{V}_i$	Local validation set of client i
$\mathcal{D}_{\text{val}}$	Global or held-out validation data used for meta-objective evaluation
$\boldsymbol{\theta}^{(t)}$	Global QNN parameter vector at round t
$\boldsymbol{\theta}_{i,k}^{(t)}$	Local QNN parameter vector of client i at unfold step k of round t
$\boldsymbol{\theta}_{i,\star}^{(t)}$	Selected best unfolded local model of client i at round t
P	Number of trainable QNN parameters
K	Number of unfolded local optimization steps
k	Unfold-step index
$f(\mathbf{x}; \boldsymbol{\theta})$	QNN prediction function
$\mathcal{L}_i(\boldsymbol{\theta})$	Empirical local loss of client i
$\mathcal{L}_i^{\text{prox}}(\boldsymbol{\theta})$	Proximal local objective of client i
$\eta_{i,k}^{(t)}$	SPSA learning rate generated for client i at unfold step k
$\delta_{i,k}^{(t)}$	SPSA perturbation scale generated for client i at unfold step k
$\mathbf{z}_{i,k}^{(t)}$	Optimization-state feature vector used by the controller
$\boldsymbol{\phi}^{(t)}$	Shared controller parameter vector at round t
$\boldsymbol{\phi}_{\eta}^{(t)}$	Controller parameters used to generate $\eta_{i,k}^{(t)}$
$\boldsymbol{\phi}_{\delta}^{(t)}$	Controller parameters used to generate $\delta_{i,k}^{(t)}$
μ	Proximal regularization coefficient
$w_i^{(t)}$	Aggregation weight of client i at round t
$\mathcal{L}_{\text{meta}}^{(t)}$	Outer meta-loss at round t
α_{out}	Outer controller learning rate
c_{out}	Outer SPSA perturbation radius
Δ_{ϕ}	Rademacher perturbation vector for outer SPSA

Table V summarizes the main notation used throughout the paper.

APPENDIX A

PROXIMAL OBJECTIVE AND DRIFT CONTROL

This appendix clarifies the drift-control role of the proximal component in DUQFL-Prox. In heterogeneous QFL, each client initializes local training from the broadcast global model $\boldsymbol{\theta}^{(t)}$. Under non-IID data and stochastic QNN optimization, repeated local updates may move the client model far from this global reference point, producing updates that are less compatible with server aggregation.

DUQFL-Prox therefore replaces the unregularized local objective $\mathcal{L}_i(\boldsymbol{\theta})$ with

$$\mathcal{L}_i^{\text{prox}}(\boldsymbol{\theta}) = \mathcal{L}_i(\boldsymbol{\theta}) + \frac{\mu}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{(t)} \right\|_2^2, \quad (33)$$

where $\mu \geq 0$ controls the strength of proximal regularization. The first term promotes local empirical improvement, whereas the second term penalizes excessive deviation from the broadcast global model.

Formally, the gradient of the proximal objective is

$$\nabla \mathcal{L}_i^{\text{prox}}(\boldsymbol{\theta}) = \nabla \mathcal{L}_i(\boldsymbol{\theta}) + \mu \left(\boldsymbol{\theta} - \boldsymbol{\theta}^{(t)} \right). \quad (34)$$

Although DUQFL-Prox uses SPSA rather than analytic gradients, this expression shows that the proximal term introduces

a drift-correcting component toward the current global model. Thus, local training is discouraged from moving arbitrarily far from $\boldsymbol{\theta}^{(t)}$.

At unfold step k , SPSA is applied to the proximal objective:

$$\boldsymbol{\theta}_{i,k+1}^{(t)} = \mathcal{U}_{\text{SPSA}} \left(\boldsymbol{\theta}_{i,k}^{(t)}; \eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}, \mathcal{L}_i^{\text{prox}} \right). \quad (35)$$

Therefore, the local update is influenced by both client-specific loss reduction and proximity to the broadcast global parameters.

The unfold-step displacement is recorded as

$$\Delta \boldsymbol{\theta}_{i,k}^{(t)} = \boldsymbol{\theta}_{i,k+1}^{(t)} - \boldsymbol{\theta}_{i,k}^{(t)}, \quad (36)$$

with magnitude

$$\left\| \Delta \boldsymbol{\theta}_{i,k}^{(t)} \right\|_2. \quad (37)$$

This quantity provides a diagnostic measure of local movement and can be used as an optimization-state feature in subsequent unfold steps.

Overall, the proximal term does not prevent local learning. Instead, it balances local improvement with aggregation compatibility. This is particularly important in DUQFL-Prox because deep unfolding increases the flexibility of local optimization, which can otherwise increase the risk of client over-specialization under heterogeneous data.

APPENDIX B OUTER SPSA GRADIENT ESTIMATION

The shared controller in DUQFL-Prox is parameterized by $\phi^{(t)}$ and generates the SPSA learning rate and perturbation scale used during unfolded local QNN optimization. The purpose of the outer update is to refine $\phi^{(t)}$ so that the induced local optimization trajectories improve post-aggregation global performance.

At communication round t , the outer meta-loss is

$$\mathcal{L}_{\text{meta}}^{(t)} = \mathcal{L}_{\text{val}}^{\text{global}}(\theta^{(t+1)}) + \lambda_{\text{fair}}\Omega_{\text{fair}}^{(t)} + \lambda_{\text{comm}}\Omega_{\text{comm}}^{(t)} + \lambda_{\text{stab}}\Omega_{\text{stab}}^{(t)}. \quad (38)$$

The dependence of $\mathcal{L}_{\text{meta}}^{(t)}$ on $\phi^{(t)}$ is indirect: the controller determines (η, δ) , which affects local QNN training, server aggregation, and finally the global validation loss. Since this mapping is expensive and not easily differentiable, DUQFL-Prox applies an outer SPSA update.

A Rademacher perturbation vector $\Delta_{\phi} \in \{-1, +1\}^{\dim(\phi)}$ is sampled, and two perturbed controllers are formed:

$$\phi^+ = \phi^{(t)} + c_{\text{out}}\Delta_{\phi}, \quad \phi^- = \phi^{(t)} - c_{\text{out}}\Delta_{\phi}. \quad (39)$$

Evaluating the federated process under these two controllers gives $\mathcal{L}_{\text{meta}}^+$ and $\mathcal{L}_{\text{meta}}^-$. For the j -th controller parameter, the SPSA finite-difference estimate is

$$\hat{g}_{\phi,j}^{(t)} = \frac{\mathcal{L}_{\text{meta}}^+ - \mathcal{L}_{\text{meta}}^-}{2c_{\text{out}}\Delta_{\phi,j}}. \quad (40)$$

Equivalently, in vector form,

$$\hat{\mathbf{g}}_{\phi}^{(t)} = \frac{\mathcal{L}_{\text{meta}}^+ - \mathcal{L}_{\text{meta}}^-}{2c_{\text{out}}} \Delta_{\phi}^{-1}. \quad (41)$$

Since $\Delta_{\phi,j} \in \{-1, +1\}$, we have $\Delta_{\phi,j}^{-1} = \Delta_{\phi,j}$.

The controller is then updated as

$$\phi^{(t+1)} = \phi^{(t)} - \alpha_{\text{out}}\hat{\mathbf{g}}_{\phi}^{(t)}, \quad (42)$$

where α_{out} is the outer learning rate. This update does not directly overwrite client-local QNN models. Instead, it modifies the controller mapping

$$\Gamma(\mathbf{z}_{i,k}^{(t)}; \phi^{(t)}) \mapsto (\eta_{i,k}^{(t)}, \delta_{i,k}^{(t)}), \quad (43)$$

so that future local SPSA behaviour becomes more aligned with the post-aggregation global objective.

APPENDIX C STABILITY AND CONVERGENCE DISCUSSION

This appendix provides a stability-oriented interpretation of DUQFL-Prox. We do not claim a general convergence theorem for arbitrary non-convex QNN objectives, since variational quantum loss landscapes may be non-convex and affected by finite-shot measurement noise. Instead, we clarify how the proposed components are designed to improve optimization stability under heterogeneous federated data.

Let the global objective be written as

$$\mathcal{F}(\theta) = \sum_{i=1}^N p_i \mathcal{L}_i(\theta), \quad p_i = \frac{n_i}{\sum_{j=1}^N n_j}. \quad (44)$$

Here, $\mathcal{L}_i(\theta)$ denotes the local objective of client i , and p_i is its sample-size proportion. Under non-IID data, the local objectives may induce different descent directions. Therefore, a client model that improves its own local objective may still be poorly aligned with the global objective after aggregation. This mismatch is the source of client drift.

DUQFL-Prox addresses this instability through three complementary mechanisms. First, the proximal penalty

$$\frac{\mu}{2} \|\theta - \theta^{(t)}\|_2^2 \quad (45)$$

discourages excessive deviation from the broadcast global model $\theta^{(t)}$. This encourages local updates to remain within a controlled neighbourhood of the global reference point, reducing the risk of over-specialized client updates.

Second, validation-based best-unfold selection prevents the method from always uploading the final unfolded state. Instead, client i uploads

$$\theta_{i,\star}^{(t)} = \theta_{i,k_i^*}^{(t)}, \quad k_i^* = \arg \min_{k \in \{1, \dots, K\}} \mathcal{L}_{i,\text{val}}(\theta_{i,k}^{(t)}). \quad (46)$$

This mechanism is important because later unfold steps may reduce training loss while increasing validation loss or local specialization.

Third, the outer meta-objective evaluates the consequence of local optimization after aggregation:

$$\mathcal{L}_{\text{meta}}^{(t)} = \mathcal{L}_{\text{val}}^{\text{global}}(\theta^{(t+1)}) + \lambda_{\text{fair}}\Omega_{\text{fair}}^{(t)} + \lambda_{\text{comm}}\Omega_{\text{comm}}^{(t)} + \lambda_{\text{stab}}\Omega_{\text{stab}}^{(t)}. \quad (47)$$

Thus, the controller is guided by post-aggregation global behaviour rather than local training loss alone.

The proximal coefficient μ controls the trade-off between local adaptation and aggregation compatibility. A larger μ more strongly penalizes movement away from $\theta^{(t)}$, whereas a smaller μ gives clients greater freedom to adapt to their local data:

$$\text{local adaptivity} \leftrightarrow \text{global aggregation compatibility}. \quad (48)$$

Therefore, DUQFL-Prox should be interpreted as a stability-aware optimization framework rather than a method that guarantees global convergence for all QNN loss landscapes. Its purpose is to reduce harmful client drift, select more generalizable local unfolded checkpoints, and align the controller with post-aggregation validation performance. This interpretation is consistent with our experimental protocol, which reports global accuracy together with mean client test accuracy, train-test gap, fairness gap, and classification metrics.

APPENDIX D RELATION TO CLASSICAL DEEP-UNFOLDED FEDERATED LEARNING

Classical deep-unfolded federated learning and DUQFL-Prox share the same high-level principle: an iterative federated optimization process can be unfolded into structured steps, and selected components of this process can be made learnable. However, the level at which learning is introduced is different.

In classical deep-unfolded weighted averaging for FL, the unfolded process is typically applied at the server aggregation level [32]. Standard FedAvg computes

$$\mathbf{w}^{(t+1)} = \sum_{i=1}^N \frac{n_i}{\sum_{j=1}^N n_j} \mathbf{w}_i^{(t)}, \quad (49)$$

where $\mathbf{w}_i^{(t)}$ denotes the local model from client i and n_i is the corresponding local sample size. Deep-unfolded weighted averaging replaces these fixed aggregation weights with learnable coefficients:

$$\mathbf{w}^{(t+1)} = \sum_{i=1}^N \Theta_i^{(t)} \mathbf{w}_i^{(t)}, \quad (50)$$

where $\Theta_i^{(t)}$ determines the contribution of client i to the global model at round t . Thus, classical deep-unfolded FL primarily asks how client updates should be weighted during aggregation.

DUQFL-Prox addresses a different question: how each client should optimize its local QNN before aggregation. Instead of learning aggregation weights $\Theta_i^{(t)}$, DUQFL-Prox learns a controller parameterized by $\phi^{(t)}$. This controller generates unfold-specific SPSA hyperparameters:

$$\left(\eta_{i,k}^{(t)}, \delta_{i,k}^{(t)} \right) = \Gamma \left(\mathbf{z}_{i,k}^{(t)}; \phi^{(t)} \right). \quad (51)$$

The server aggregation remains sample-size-weighted:

$$\boldsymbol{\theta}^{(t+1)} = \sum_{i \in \mathcal{S}^{(t)}} w_i^{(t)} \boldsymbol{\theta}_{i,*}^{(t)}. \quad (52)$$

The key difference is that the uploaded model $\boldsymbol{\theta}_{i,*}^{(t)}$ is produced through controller-guided deep-unfolded proximal QNN optimization and validation-based best-unfold selection.

Therefore, DUQFL-Prox transfers the deep-unfolding principle from server-side aggregation-weight learning to client-side quantum optimizer-policy learning. This distinction is important because QFL introduces optimization challenges not present in ordinary classical FL, including finite-shot measurement noise, SPSA perturbation sensitivity, and non-convex variational quantum loss landscapes. DUQFL-Prox is therefore complementary to classical deep-unfolded FL methods: a future extension could jointly learn client-side QNN optimization policies and server-side aggregation weights, whereas this work focuses on stabilizing local QNN optimization before aggregation.

APPENDIX E

ADDITIONAL ABLATION AND CONTROLLER TRACE ANALYSIS

A. Compared Methods

We compare DUQFL-Prox with representative QFL baselines and ablation variants. All methods use the same QNN architecture, dataset split, client partitioning, number of communication rounds, and evaluation protocol for each dataset. Thus, differences in performance are attributed to the local optimization strategy, checkpoint-selection mechanism, and

drift-control design rather than to model capacity or data partitioning. Table VI summarizes the role of each method.

Although DUQFL-drift and DUQFL-Prox both include drift-control mechanisms, they regularize drift differently. DUQFL-drift uses an explicit displacement constraint, whereas DUQFL-Prox uses a proximal penalty that continuously discourages deviation from the broadcast global model. Similarly, FedProx-QFL and DUQFL-Prox both use proximal regularization, but at different levels: FedProx-QFL regularizes a conventional local optimization routine, whereas DUQFL-Prox embeds proximal regularization within a controller-guided unfolded QNN optimization trajectory.

B. Effect of Best-Unfold Selection and Proximal Drift Control

Figure 9 presents the ablation study on the Genome non-IID setting. The comparison isolates the contribution of the main DUQFL-Prox components: controller-guided unfolding, validation-based best-unfold selection, and drift control. DUQFL-last uploads the final unfolded local state, DUQFL-best uploads the validation-preferred unfolded checkpoint, DUQFL-drift adds explicit displacement control, and DUQFL-Prox incorporates proximal drift regularization.

The results show that DUQFL-last improves over Default-QFL in some rounds, but its client-level performance remains less stable than DUQFL-best and DUQFL-Prox. This indicates that the final unfolded state is not necessarily the most generalizable checkpoint under heterogeneous client data. Validation-based best-unfold selection improves this behaviour by preventing over-aggressive or poorly generalizing later unfold states from being uploaded to the server.

Adding drift control further improves stability. DUQFL-Prox achieves the highest final mean client test accuracy, the lowest train-test gap, and the smallest client fairness gap among the DUQFL variants. These trends suggest that proximal regularization improves aggregation compatibility by limiting excessive client movement, while best-unfold selection improves the quality of the checkpoint selected for aggregation.

The comparison with FedProx-QFL is also important. Although both FedProx-QFL and DUQFL-Prox include proximal regularization, FedProx-QFL applies it to a standard local optimization process. In contrast, DUQFL-Prox applies proximal regularization inside an adaptive unfolded QNN optimization trajectory whose SPSA learning rate and perturbation scale are generated by a controller. Therefore, the improvement of DUQFL-Prox over FedProx-QFL indicates that proximal regularization alone is insufficient; stable heterogeneous QFL also requires adaptive control of the local QNN optimization process before aggregation.

APPENDIX F

ADDITIONAL CONTROLLER TRACE ANALYSIS

This appendix provides trace-level diagnostics for DUQFL-Prox. While the main ablation study evaluates global and client-level performance, the following figures examine the internal behaviour of the controller during unfolded local QNN optimization. These diagnostics support the claim that

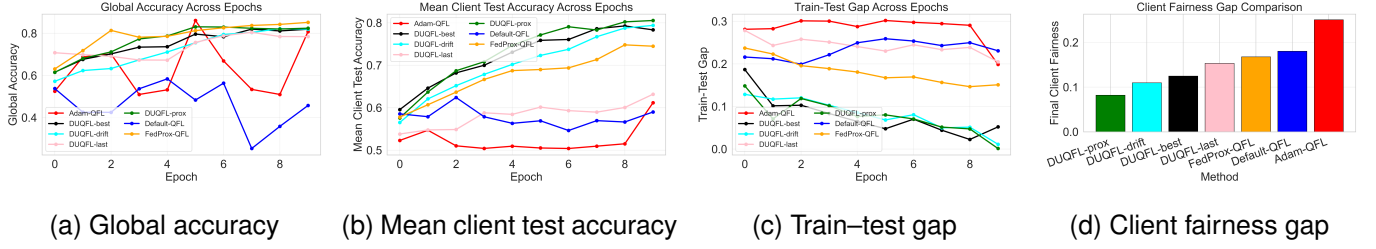


Fig. 9: Ablation analysis of DUQFL variants on the Genome non-IID setting. DUQFL-last uploads the final unfolded local state, DUQFL-best uploads the validation-preferred unfolded checkpoint, DUQFL-drift adds explicit displacement control, and DUQFL-Prox incorporates proximal drift regularization. DUQFL-Prox achieves the strongest client-level generalization, lowest train-test gap, and smallest fairness gap, indicating that best-unfold selection and proximal drift control improve stability and balance under heterogeneous QFL.

TABLE VI: Interpretation of QFL baselines and DUQFL ablation variants.

Method	Deep unfold	Controller	Best unfold	Drift control	Role
Default-QFL	×	×	×	×	Baseline QFL using Qiskit's default SPSA optimizer and FedAvg aggregation.
Adam-QFL	×	×	×	×	Generic adaptive optimizer baseline used to assess whether standard local adaptivity is sufficient under QFL heterogeneity.
FedProx-QFL	×	×	×	✓	Proximal QFL baseline that tests drift control without deep unfolding or controller-guided SPSA adaptation.
DUQFL-last	✓	✓	×	×	Tests whether uploading the final unfolded local checkpoint is sufficient.
DUQFL-best	✓	✓	✓	×	Tests validation-based best-unfold checkpoint selection.
DUQFL-drift	✓	✓	✓	✓	Tests explicit displacement-based drift control within unfolded local QNN optimization.
DUQFL-Prox	✓	✓	✓	✓	Full proposed method combining controller-guided unfolded SPSA, best-unfold selection, and proximal drift regularization.

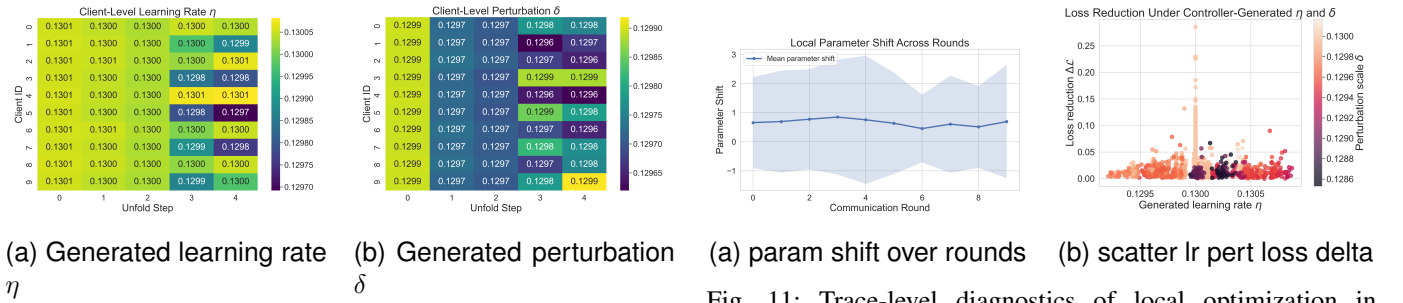


Fig. 10: Client- and unfold-level controller behaviour in DUQFL-Prox. The heatmaps show that the generated SPSA learning rate and perturbation scale vary across clients and unfold steps, supporting the claim that DUQFL-Prox performs controller-guided local optimization rather than using a fixed SPSA schedule.

DUQFL-Prox does not use a fixed SPSA schedule, but instead generates client- and unfold-dependent optimization behaviour.

Figure 10 reports the learning rate η and perturbation scale δ generated by the controller for each client and unfold step. The heatmaps show that the generated SPSA hyperparameters vary across clients and unfold steps, indicating that the controller adapts local optimizer behaviour according to the optimization state rather than applying a single fixed schedule.

Figure 11 provides two complementary diagnostics. The parameter-shift curve monitors local movement across communication rounds, whereas the loss-reduction scatter plot relates controller-generated hyperparameters to local loss improve-

Fig. 11: Trace-level diagnostics of local optimization in DUQFL-Prox. The parameter-shift curve monitors local movement during unfolded optimization, while the scatter plot shows that controller-generated SPSA hyperparameters remain within a stable numerical range and produce positive local loss reduction for most client-unfold updates.

ment. Most client-unfold updates yield positive loss reduction while the generated learning rates and perturbation scales remain within a stable numerical range. This supports the interpretation that the controller performs fine-grained adaptation without inducing unstable hyperparameter excursions.

Figure 12 further supports the validation-based checkpointing mechanism. The normalized unfold-step analysis shows that the largest average loss reduction often occurs during early unfold steps, while the selected-unfold distribution confirms that the final unfolded state is not always selected for upload. Therefore, best-unfold selection is necessary to prevent later, less generalizable unfold states from dominating the server aggregation.

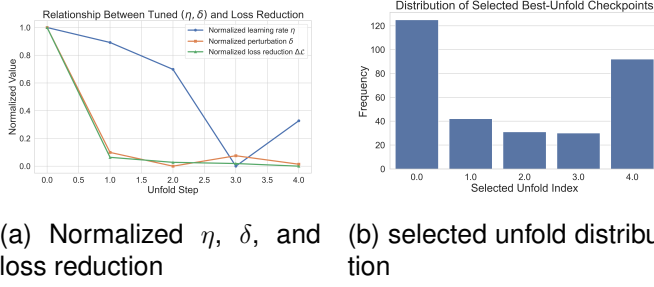


Fig. 12: Unfold-level behaviour of DUQFL-Prox. The normalized plot shows that the largest local loss reduction often occurs during early unfold steps, while the selected-unfold distribution confirms that the final unfolded state is not always the checkpoint uploaded for aggregation. These diagnostics support the use of validation-based best-unfold selection.

A. Trace Logging and Reproducibility

To support reproducibility and detailed ablation analysis, each experimental run saved round-level, client-level, and classification-level outputs. Round-level CSV files stored global accuracy, validation loss, client accuracy statistics, meta-loss, fairness information, and timing. Client-level trace files stored the unfold index, generated learning rate, perturbation scale, local losses, local train/test accuracy, validation loss, parameter displacement, clipping indicator, selected unfold index, client size, and heterogeneity value. These trace logs make it possible to verify whether the controller changes the local SPSA behaviour across clients, unfold steps, and communication rounds.

The main saved outputs were:

- `global_accuracies.csv`: global and client train/test accuracies across communication rounds;
- `validation.csv`: validation loss across rounds;
- `client_trace.csv`: unfold-level client traces, including generated η , generated δ , validation loss, parameter shift, and selected unfold index;
- `outer_meta.csv`: outer SPSA controller-update information, including perturbed meta-loss values and controller-gradient norm;
- `classification_metrics.csv`: precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, and confusion-matrix values;
- `global_params.npz`: saved global QNN parameter vectors across communication rounds.

The result files were organized by method, dataset, split type, random seed, and timestamp. This structure allows each run to be regenerated from the stored configuration and output files.

APPENDIX G

IBM QUANTUM HARDWARE WORKLOAD EVIDENCE

To assess practical deployability, selected trained global QNN checkpoints were executed on real IBM quantum hardware. This experiment was performed as post-training hardware validation rather than full hardware-in-the-loop federated training. The saved global checkpoints were assigned to the

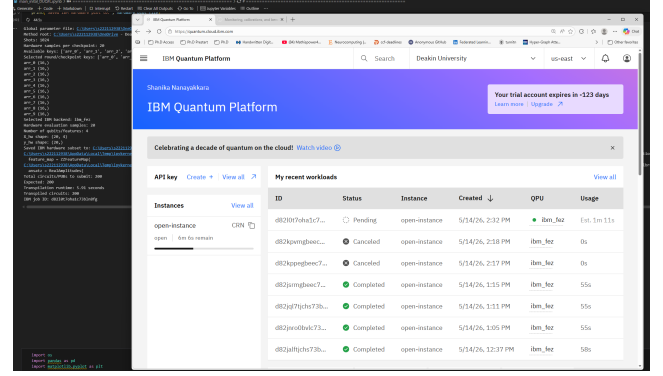


Fig. 13: IBM Quantum Platform workload evidence for real-hardware execution of trained QFL checkpoints. The workload history shows completed executions on the `ibm_fez` backend using finite-shot measurement. This screenshot is included as execution evidence only; the quantitative simulator–hardware accuracy comparison is reported separately.

trained QNN circuits, transpiled for the selected IBM backend, and executed using finite-shot measurement.

Figure 13 provides IBM Quantum Platform workload evidence for the hardware execution. The workload history shows completed executions on the `ibm_fez` backend using the IBM Quantum open instance. This screenshot is included only as execution evidence; the quantitative simulator–hardware accuracy comparison is reported separately in the main experimental results.